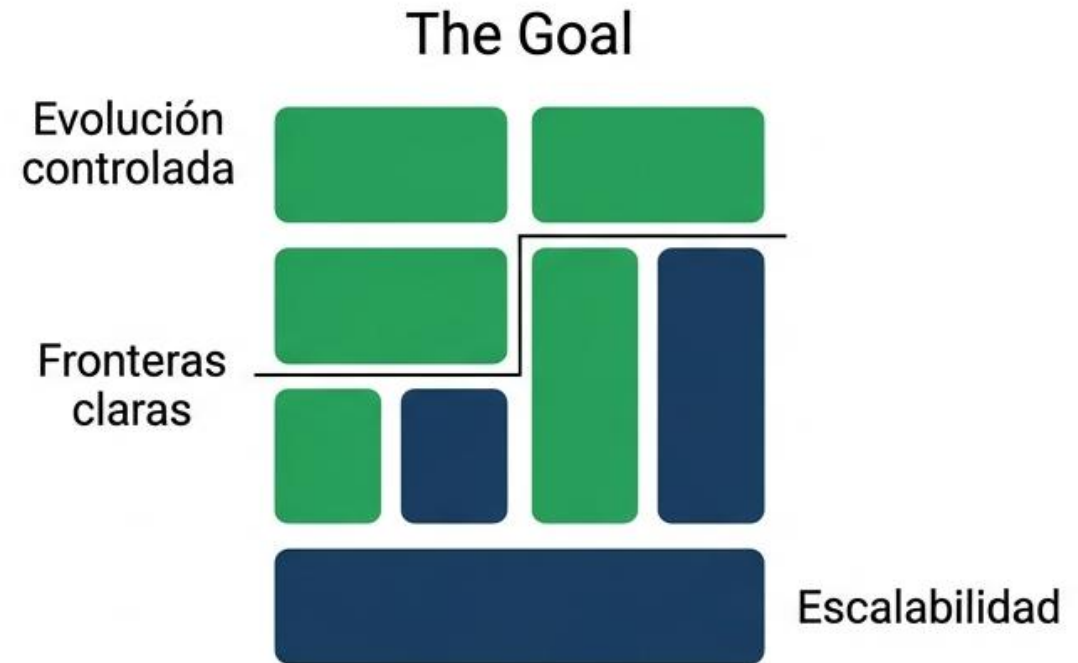
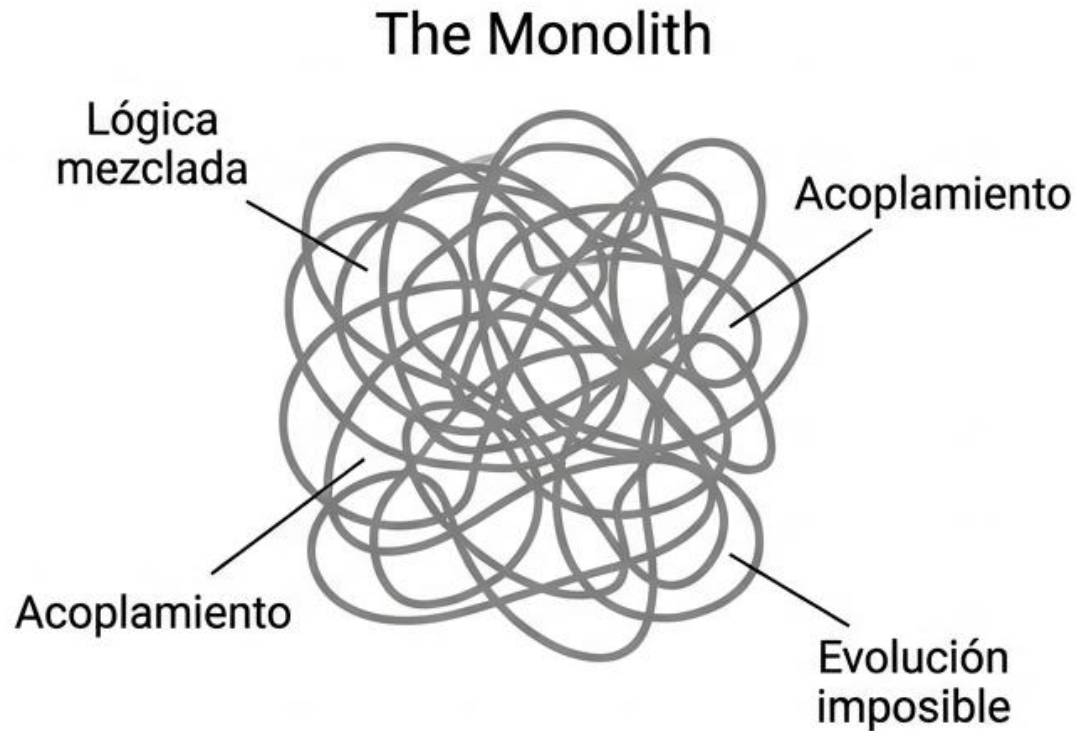


Refactorización de Monolitos: Guía de Implementación de Arquitectura Hexagonal.



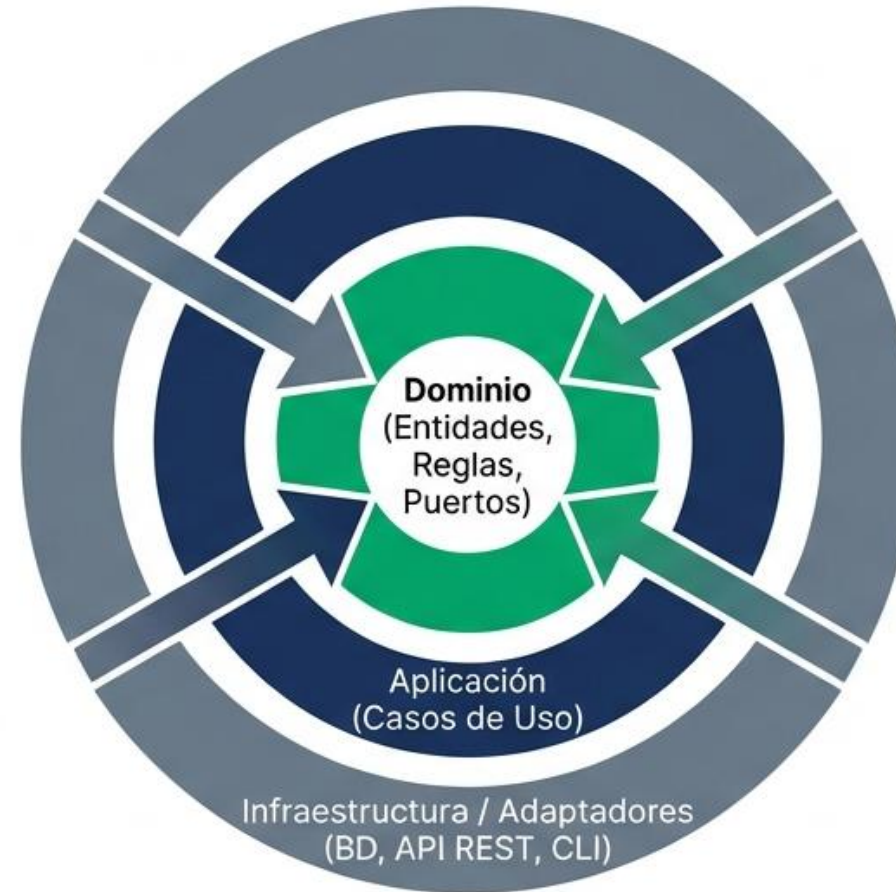
El Verdadero Trabajo del Ingeniero de Software

La mayor parte de la carrera profesional no consiste en diseñar sistemas desde cero, sino en evolucionar sistemas existentes ahogados en deuda técnica.



La Meta: Arquitectura Hexagonal (Ports & Adapters)

Un diseño donde la lógica de negocio es el centro absoluto y la infraestructura es un detalle intercambiable. La regla inquebrantable: Las dependencias siempre apuntan hacia adentro.



Roadmap de Transformación

Refactorizar un monolito a microservicio requiere disciplina. El trabajo se divide en cinco fases secuenciales estrictas.

1

Análisis:

Mapa de Deuda
Técnica.

2

Corazón:

Dominio y
Puertos.

3

Orquestación:

Capa de
Aplicación.

4

Mundo Exterior:

Entradas y
Evolución.

5

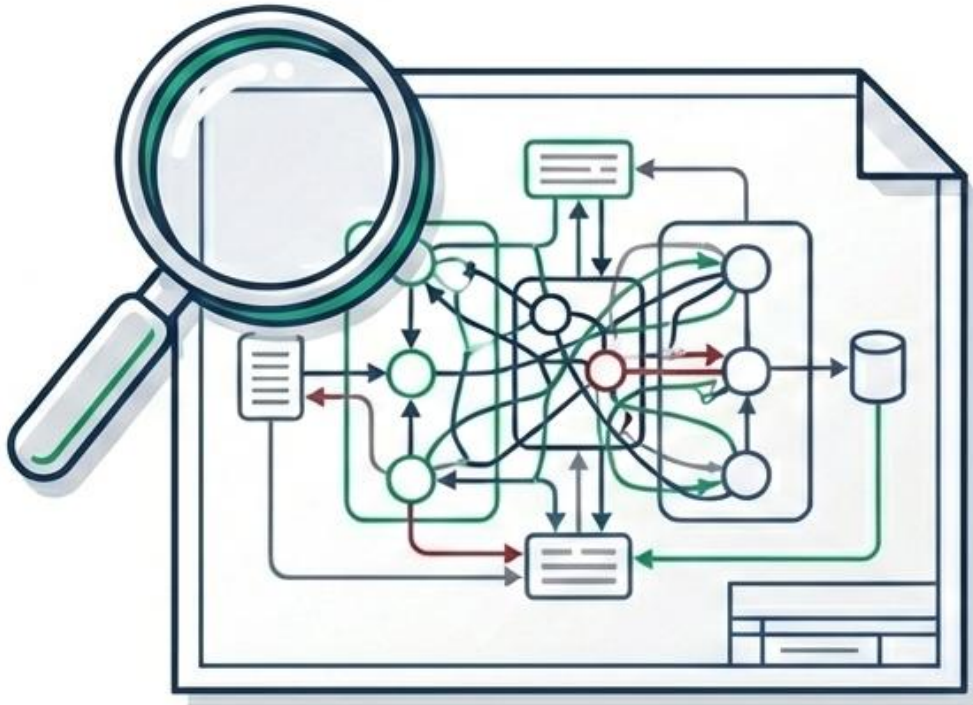
Cierre:

Consolidación
y Defensa.



Fase 1: Análisis del Monolito y Mapa de Deuda Técnica

Comprender el sistema actual y documentar sus problemas arquitectónicos antes de tocar una sola línea de código.



Actividades y Entregables

- Crear diagrama de dependencias del monolito actual.
- Identificar violaciones con número de línea (ej. SQL en controladores).
- Documento de 5 páginas mínimo.



Definition of Done

Checklist de Salida:

Puedes explicar en voz alta (sin leer) cuál es el problema principal de acoplamiento y cuál sería el primer paso para separarlo.



Fase 2: El Corazón del Sistema (Dominio)

Construir las entidades, reglas y puertos sin ninguna dependencia externa. Debe poder ejecutarse sin internet, base de datos, ni framework web.



Actividades:

- Entidades con Pydantic, Objetos de Valor, Excepciones claras.
- Interfaces ABC nombradas por negocio.
- 0 violaciones arquitectónicas.



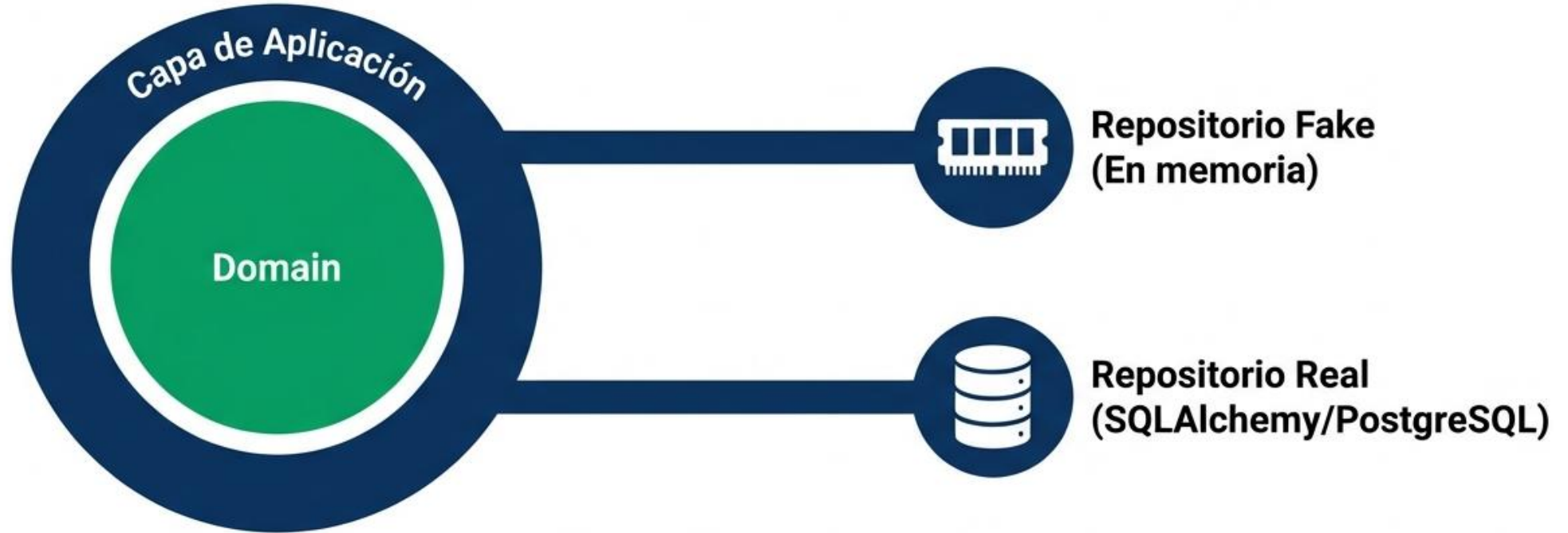
Checklist de Salida:

100% de las pruebas unitarias (mínimo 20) pasan en milisegundos sin levantar infraestructura.



Fase 3: Orquestación y Adaptadores de Salida

Implementar casos de uso que orquesten el flujo (Comando → Cargar Entidad → Regla de Dominio → Persistir) y conectar el almacenamiento.



Checklist de Salida:

- Los mismos casos de uso funcionan con el repositorio en memoria y con el repositorio real cambiando solo la configuración.



Fase 3: Orquestación y Adaptadores de Salida

- Arquitectura Hexagonal
- Explicación detallada de la fase donde la aplicación coordina los casos de uso y se conecta con los mecanismos de persistencia.



Idea Central de la Fase

- En esta fase se implementa la capa de aplicación.
- Su responsabilidad principal es orquestar el flujo de ejecución de los casos de uso del sistema y conectar el dominio con los mecanismos de persistencia o infraestructura.



Flujo de Ejecución del Caso de Uso

- Un caso de uso sigue el siguiente flujo:
- Comando → Cargar Entidad → Aplicar Regla de Dominio → Persistir
- Este flujo representa cómo la aplicación coordina la lógica del sistema.



Paso 1: Comando

- El sistema recibe una solicitud.
- Esta solicitud puede venir desde una API, interfaz web, CLI o cualquier interfaz externa.
- El comando representa la intención del usuario o sistema externo.



Paso 2: Cargar Entidad

- El caso de uso obtiene los datos necesarios del dominio.
- Esto se hace utilizando un repositorio.
- El repositorio permite acceder a los datos sin que el dominio dependa directamente de la base de datos.



Paso 3: Regla de Dominio

- La entidad del dominio ejecuta la lógica de negocio.
- Las reglas de dominio representan el conocimiento del negocio.
- Estas reglas deben estar aisladas de la infraestructura.



Paso 4: Persistir

- Después de aplicar la lógica de negocio, el resultado debe almacenarse.
- Esto se realiza nuevamente a través del repositorio.
- La aplicación no conoce cómo se almacenan los datos, solo utiliza la interfaz.



Relación entre Capas

- En el centro se encuentra el Domain.
- El dominio contiene las reglas de negocio puras.
- Alrededor está la capa de aplicación que coordina los casos de uso.



Desacoplamiento de Infraestructura

- La aplicación no depende directamente de la base de datos.
- En su lugar utiliza interfaces llamadas puertos.
- Estas interfaces permiten conectar diferentes implementaciones.



Adaptadores de Salida

- Los adaptadores de salida implementan los puertos definidos por la aplicación.
- Permiten conectar el sistema con diferentes tecnologías de almacenamiento.



Repositorio Fake (En memoria)

- Implementación utilizada principalmente para pruebas.
- Los datos se almacenan en memoria.
- Permite ejecutar los casos de uso sin necesidad de una base de datos real.



Repositorio Real (SQLAlchemy / PostgreSQL)

- Implementación real del repositorio.
- Se conecta con una base de datos real.
- Utiliza tecnologías como SQLAlchemy y PostgreSQL.



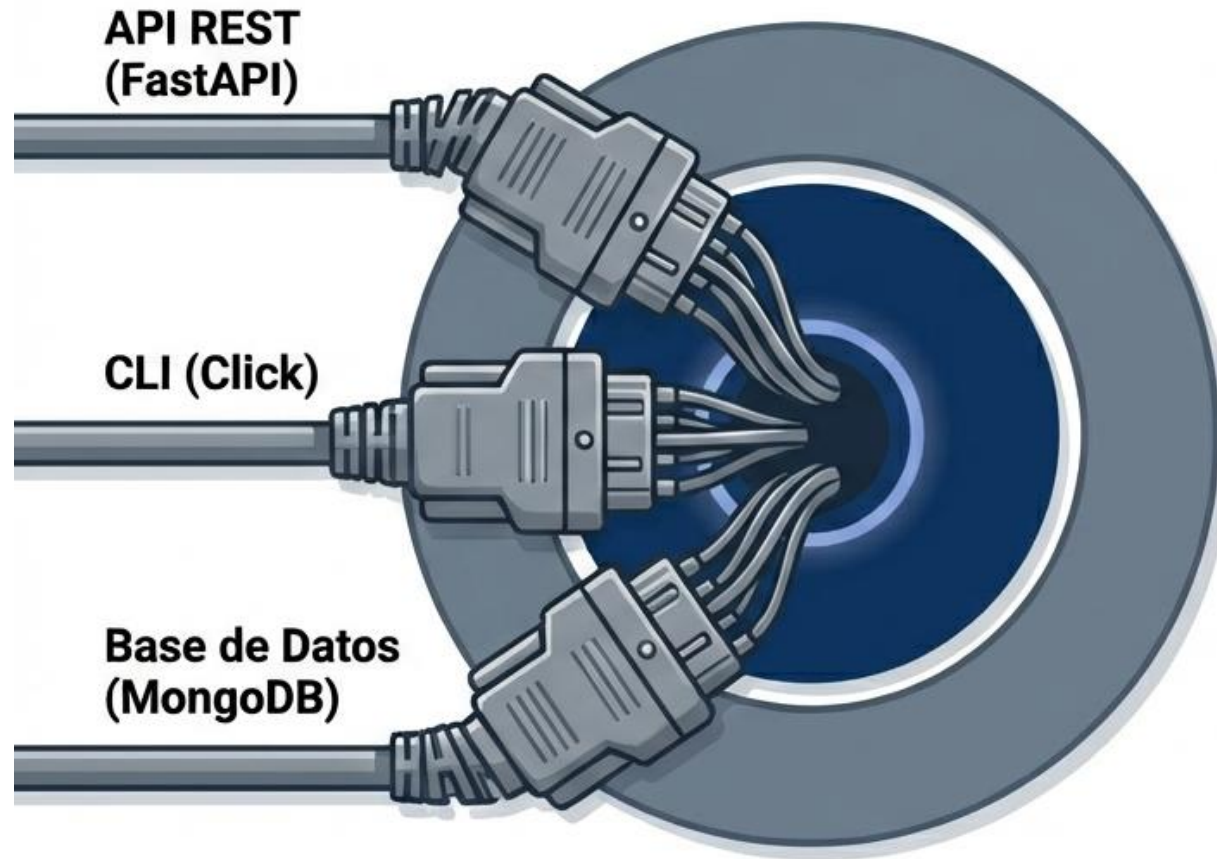
Beneficio Clave de la Arquitectura

- Los mismos casos de uso funcionan con diferentes repositorios.
- Solo cambia la configuración del sistema.
- No es necesario modificar el dominio.



Fase 4: El Mundo Exterior y la Prueba de Fuego

Conectar el núcleo con el mundo exterior y demostrar la capacidad de evolución agregando canales sin romper el dominio.



Actividades y Evolución

- Replicar endpoints originales con FastAPI + 2 nuevos.
- Agregar un 2do canal de entrada (CLI, Webhooks, RabbitMQ) o un 2do adaptador de salida (MongoDB, Redis).

Checklist de Salida:

Un commit demostrando un nuevo adaptador agregado con CERO líneas modificadas en las entidades del dominio.



Fase 5: Consolidación, Pruebas y Cierre

Orden, evidencia y comunicación técnica de alto nivel.



Repositorio GitHub



Cobertura CI/CD



Manual Técnico & Diagramas C4

Documentación: README claro para ejecutar `MODO_REPOSITORIO=memoria` y `MODO_REPOSITORIO=postgres (Docker)`.



Checklist de Salida: Cualquier compañero puede clonar el repositorio, seguir el README y ejecutar el proyecto sin adivinar.



Antipatrones Comunes: El Controlador

Evita fugar lógica de negocio hacia la capa de infraestructura.

Antes ❌

```
if monto > limite_diario:  
    raise ValueError
```

❌ **Lógica de negocio en API:**
El controlador toma decisiones.

Después ✅



✅ **Controlador como Traductor:**
Las reglas viven en la Entidad.
El controlador solo traduce.



El Estándar de Entrega (Repositorio e Informe)

La calidad de la entrega debe reflejar el nivel de un ingeniero de software senior.

Entregable 1: Git Repo

- 2 ramas: ``main`` (original) y ``hexagonal`` (refactorizada).
- Archivos OBLIGATORIOS:
 - ``COMPARATIVA.md`` (métricas antes/después).
 - ``ARQUITECTURA.md``.
- Historial de commits limpio reflejando las 5 fases.

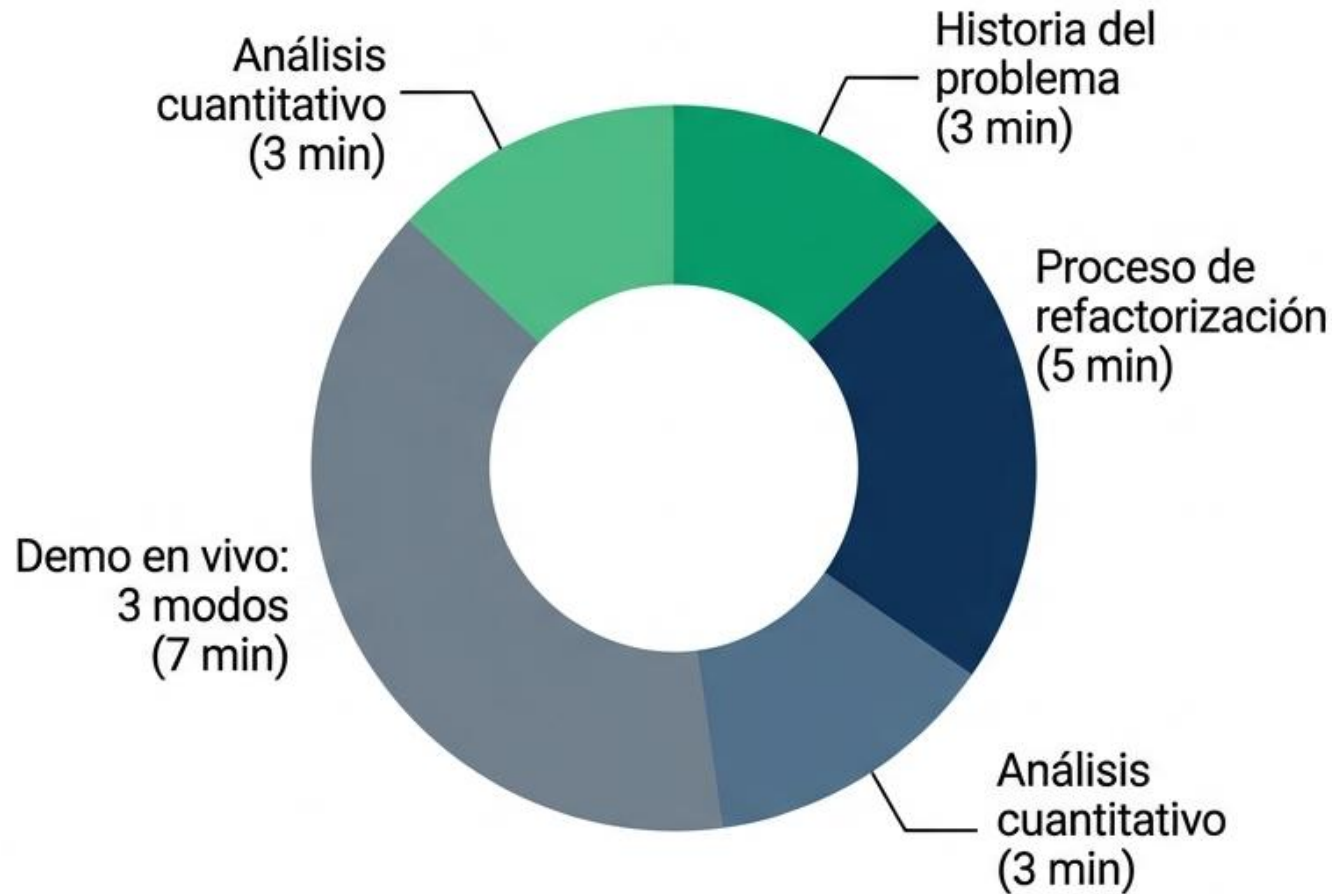
Entregable 2: Informe Técnico (PDF)

- Mínimo 30 páginas.
- Decisiones justificadas ('POR QUÉ, no QUÉ').
- Catálogo completo de Puertos.
- Evidencia de evolución sin tocar dominio.
- Reflexión crítica fundamentada.



Sustentación y Demo en Vivo

Estructura de la presentación de 20 minutos y opciones para puntos extra.



Entregable 4 (Opcional, +5 pts)

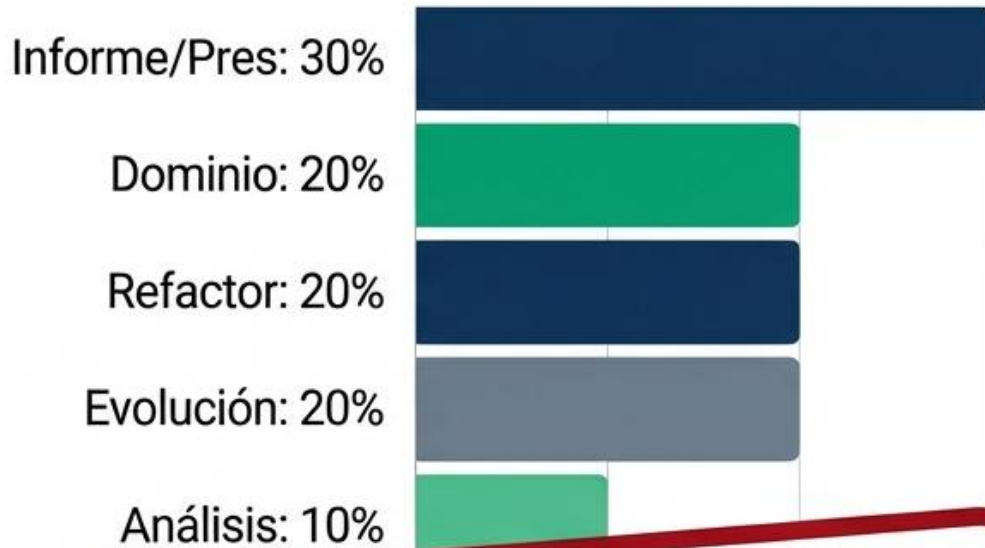
Video (10 min max)
demostrando:

- Tests en < 2 segs.
- Arranque con docker - compose.
- 2do canal de entrada funcionando.



Rúbrica y Defensa del Diseño

Prepárense para fundamentar cada línea de código basada en decisiones arquitectónicas.



Penalización Crítica: -15%
si no puedes responder
sobre tu propio código.

Preguntas Esperadas en la Defensa:

- ¿Diferencia entre Puerto de Entrada y Adaptador de Entrada?
- ¿Si migramos a MongoDB, qué archivos exactos cambian?
- ¿Cómo se convertiría esto en un microservicio independiente?

